KTH ROYAL INSTITUTE
OF TECHNOLOGY

Degree Project in Electrical Engineering

Second cycle, 30 credits

Reverse Engineering of Deep Learning Models by Side-Channel Analysis

XUAN WANG

Stockholm, Sweden, 2023

Reverse Engineering of Deep Learning Models by Side-Channel Analysis

XUAN WANG

Master's Programme, Embedded Systems, 120 credits
Date: January 30, 2023

Supervisor: Huanyu Wang
Examiner: Elena Dubrova
School of Electrical Engineering and Computer Science
Swedish title: Omvänd konstruktion av djupinlärningsmodeller genom sidokanalanalys

Abstract

Side-Channel Analysis (SCA) aims to extract secrets from cryptographic systems by exploiting the physical leakage acquired from implementations of cryptographic algorithms. With the development of Deep Learning (DL), a new type of SCA called Deep Learning Side-Channel Analysis (DLSCA) utilizes the advantages of DL techniques in data features processing to break cryptographic systems more efficiently. Recent works show that DLSCA could be applied not only to extract the secrets from implementations of cryptographic algorithms but also to reverse the architecture of neural networks from physical devices. In short, reverse engineering of neural networks aims to recover the parameters and architecture of neural networks by accessing only the input and output of the target model. However, most of previous works have focused on recovering the ratio between weights and biases, which is not threatening enough.

This thesis aims to explore to which extent the DLSCA can make reverse engineering on neural networks more efficient. To achieve this goal, we first implement perceptron networks as the target model on an 8-bit Atmel ATXmega128D4 microcontroller. Then, we use Chipwhisperer Lite to capture power traces from the victim device during the execution of the neural network. Our experimental results first show that it is feasible to distinguish different activation functions directly from the captured traces. Afterward, we select the multiplication of the weights and inputs as the attack point, and use Test Vector Leakage Assessment (TVLA) method to detect the location of leakage intervals. Next, we train a DL classifier by using the captured traces and use the trained model to classify the actual weight of the target neural network. We show that it is feasible to reverse a weight by using less than 1000 traces on average. For some specific weights, our DL classifier is able to recover them by using only one trace.

Keywords

Side-Channel Attack, Deep Learning, Reverse Engineering, Perceptron Neural Network

Sammanfattning

Sidokanalanalys (SCA) syftar till att extrahera hemligheter från kryptografiska system genom att utnyttja det fysiska läckaget från implementeringar av kryptografiska algoritmer. Med utvecklingen av Djup lärning (DL), använder en ny typ av SCA kallad DLSCA fördelarna med DL-tekniker i datafunktionsbehandling för att bryta kryptografiska system mer effektivt. Nya arbeten visar att DLSCA inte bara kan användas för att extrahera hemligheterna från implementeringar av kryptografiska algoritmer utan också för att vända arkitekturen för neurala nätverk från fysiska enheter. Kort sagt, omvänd konstruktion av neurala nätverk syftar till att återställa parametrarna och arkitekturen för neurala nätverk genom att endast komma åt ingången och utdata från målmodellen. Men de flesta tidigare arbeten har fokuserat på att återställa förhållandet mellan vikter och fördomar, vilket inte är tillräckligt hotfullt.

Denna avhandling syftar till att undersöka i vilken utsträckning DLSCA kan göra reverse engineering på neurala nätverk mer effektiv. För att uppnå detta mål implementerar vi först perceptronnätverk som målmodell på en 8-bitars Atmel ATXmega128D4 mikrokontroller. Sedan använder vi Chipwhisperer Lite för att fånga kraftspår från offrets enhet under exekveringen av det neurala nätverket. Våra experimentella resultat visar först att det är möjligt att särskilja olika aktiveringsfunktioner direkt från de fångade spåren. Efteråt väljer vi multiplikationen av vikterna och ingångarna som attackpunkt och använder TVLA-metoden för att upptäcka var läckageintervallen finns. Därefter tränar vi en DL-klassificerare genom att använda de fångade spåren och använder den tränade modellen för att klassificera den faktiska vikten av det neurala målnätverket. Vi visar att det är möjligt att vända en vikt genom att använda mindre än 1000 spår i genomsnitt. För vissa specifika vikter kan vår klassificerare DL återställa dem genom att bara använda ett spår.

Keywords

Sidokanalattack, Djup lärning, Omvänd Konstruktion, Perceptron Neurala Nätverk

Acknowledgments

I would like to thank the Royal Institute of Technology for the opportunity I had to attend a Master's Degree in this renowned university.

I would also express my sincere gratitude to my examiner, Professor Elena Dubrova, who offered me a Master's Thesis project from the beginning and gave me a lot of help and support during these six months. Without her valuable advice and guidance on my thesis, it would not be possible for me to complete the thesis smoothly.

A lot of gratitude goes to Huanyu, my supervisor, who has offered me valuable suggestions in my academic studies. When I had some problems with my project, he always spent his precious time helping me to solve the problems.

I would also like to thank my girlfriend, Shuhui, who gave me many companions when I was lost during the project. Her encouragement gave me forward momentum.

A special thank also goes to Yanning and Weiyao, who offered me a lot of help during the project. For example, when I had no idea about the current status, they always gave me a precise analysis.

My gratitude goes to my family for all their encouragement and support during this period.

Stockholm, January 2023
Xuan Wang

Contents

List of Figures

List of Tables

Chapter 1

Introduction

With the rapid development of computer science and information technology, information security has become an important challenge. Cryptography [10] is a vital subject that plays a critical role in information security in our daily lives. In a cryptographic system, the secret key and the cryptographic algorithm protect the confidentiality, integrity, and availability of information theoretically. Nevertheless, in the real world, a cryptographic algorithm relies on hardware, especially some chips, to provide an execution platform. There is a risk of information leakage [24] when the cryptographic system is executed on a physical device. The available information leakage contains execution time [22], power consumption [21], Electromagnetic (EM) radiation [19], acoustic information [13], cache information [36], etc. The attacker can acquire the secret information by analyzing the leakage information. This attack method is called Side-Channel Analysis (SCA).

In most cases, traditional SCA is used to analyze the secret key of the cryptographic system by observing and analyzing the leakage information directly. Divergent from traditional SCA, the Deep Learning Side-Channel Analysis (DLSCA) trains a Deep Learning (DL) classifier to find the correlation between leakage information and the secret key. The dataset of the classifier could be some power traces captured when the cryptographic algorithm system is being executed. Several works compared a traditional SCA, called template attack, with the DLSCA [23][20]. Combining with DL, the attacker can obtain the secret key with less leakage information than traditional SCA [40]. Hence, this new type of SCA increases the efficiency of the attack dramatically. In recent works, SCA has been used combined with different DL models, including Multilayer Perceptron (MLP) [25][11] and Convolutional Neural Network (CNN) [11] [38].

DLSCA could be applied to not only cryptographic systems but also reverse engineering of neural networks [17]. In ordinary cases, when considering distinct companies, it is possible to relate the specific details of a neural network, such as the number of neural network layers, activation function, and loss function. In addition, the training data set used to train neural networks may reveal private information. Besides, patients' genotypes are often applied as a fundamental support for machine learning models, which are used for medical treatments making them extremely sensitive [12]. Despite privacy issues, recovering the neural network parameters can help to obtain business secrets without violating intellectual property rights [3]. Consequently, obtaining the layout and parameters of a neural network becomes a target for attackers to seek profit. To protect the users' rights and privacy information, a better understanding of reverse engineering for the neural network is required.

The reverse engineering of neural networks is to reverse the internal architecture of the neural network only by inputs and outputs. These internal details could be the number of layers, the type of activation functions, weights, and the bias. Recent works show that there is some progress in reverse engineering of neural networks not only by traditional methods but also by SCA. Fault injection attacks are applied to reverse engineering of neural networks [6]. Furthermore, Hua et al. tried to use the timing and memory SCA method to reverse two types of CNN models, namely AlexNet and SqueezeNet [17]. However, there are some limitations in their experiments. On the one hand, only the ratio of the weight and bias could be reversed, but not for the weight itself. On the other hand, they require access to the original training dataset. In real life, the training datasets are extremely sensitive for some companies. For instance, if the neural network is trained on medical records [14], the confidentiality of the training dataset should be preserved by non-authorized parties.

1.1 Contributions and Organizations

The main contributions of our experiments are as follows:

1. Firstly, we eliminate the requirement of the original dataset. We show that it is possible to only use the power consumption traces captured during the execution of the model to reverse the actual weight of the target neural network.

2. Secondly, we show that it is feasible to distinguish the features of different activation functions from the captured traces

3. Besides, the Test Vector Leakage Assessment (TVLA) method is used to detect the location of leakage information.

4. Furthermore, we focus on recovering the weight, not only the ratio of weight and bias. The primary approach is to train a DL classifier for the hamming weight of the multiplication of inputs and weights.

5. All the experiments are conducted on an 8-bit Atmel ATXmega128D4 microcontroller. We use the Chipwhisperer Lite for measuring the traces.

1.2 Delimitation

The delimitation is shown as follows:

1. We capture power traces by using a Chipwhisperer Lite from an 8-bit Atmel ATXmega128D4 microcontroller. Hence, there is a restriction of the target neural network that all the parameters of the target neural network must be 8 bits. These parameters contain inputs, weights, and biases.

2. The max buffer size is to cache 24537 samples. For the target neural network, the traces could be captured within 24000 samples. If the target neural network is an MLP or CNN model, the length of the trace may exceed the max buffer size.

1.3 Outline

The thesis is organized as follows:

- Chapter2 describes the background of deep learning and reverse engineering.

- Chapter3 first shows the target neural network architecture, traces acquisition, attack point, and power model selection. Then, it illustrates the leakage detection process, DL classifier training, and evaluation methods.

- Chapter4 gives the results of activation function recognition, leakage detection, and model architecture reversing.
- Finally, Chapter5 summarizes the thesis and provides potential future work of the project.

Chapter 2

Theoretical Background

In this chapter, firstly, we give details about the deep learning theory utilized in our thesis project. Then the related technology knowledge of reverse engineering is introduced.

2.1 Deep Learning

With the rapid development of computer science and information technology, Artificial Intelligence (AI) has become increasingly popular in our daily life. Generally speaking, the current AI which can help humans to accomplish some specific tasks is called narrow AI [2]. AI takes various forms in many aspects, such as an intelligent assistant Siri [32], and the super-machine Watson [27].

The most popular approach to implementing and accomplishing AI is called Machine Learning (ML). It is used in a wide variety of aspects of AI, such as computer vision [8] and speech recognition [35]. The primary method of ML is induction rather than deduction. It can resolve and learn some disciplines from massive data and predict events in real life.

DL is a type of ML. Traditional ML can learn some regular patterns from massive data by some algorithms. While DL can analyze and learn from the dataset by a neural network. There are two kinds of DL models which contain supervised and unsupervised DL models. For instance, CNN [33] is a supervised DL model, and Deep Belief Nets (DBNs) [26] is an unsupervised model.

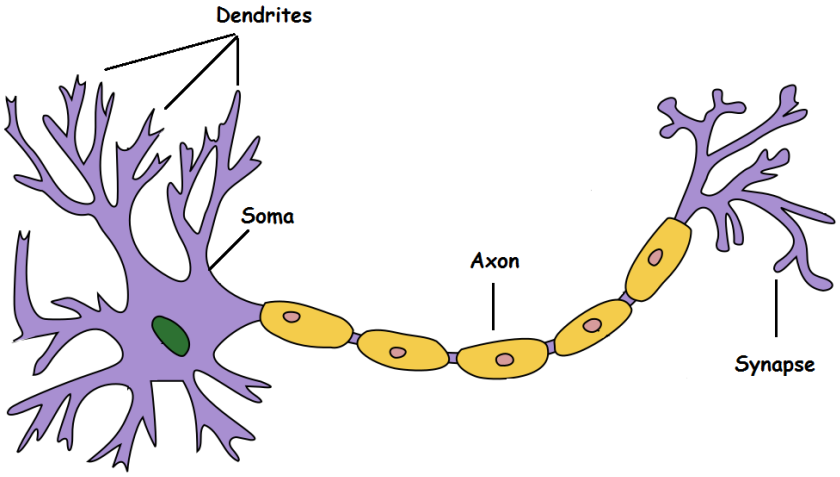

Figure 2.1: The Architecture of a Biological Neuron [7]

2.1.1 Artificial Neural Network

An artificial neural network is the kernel of the DL algorithm, which can learn by imitating human nerves. The basic unit of the artificial neural network is a number of connected nodes called artificial neurons and is sourced from the biological neuron. Figure 2.1 shows the architecture of the biological neuron, which consists of dendrites, soma, axon, and synapse.

In a neural network, the node is used to transmit signals and information, which can be real numbers in realistic models. The output of the neuron is calculated by a linear or non-linear function with inputs, weights, and bias. In layman's terms, the weight is a parameter representing the neuron's importance in the whole neural network. The bias is a constant number to improve the fitting ability of the neural network.

2.1.2 Multilayer Perceptron

Perceptron is a linear binary classifier which is first invented by Frank Rosenblatt [31]. It consists of only one layer Threshold Linear Unit (LTU), which is proposed by Warren McCulloch and Walter Pitts [15]. The input and output of LTU are real numbers rather than binary codes. Each input must connect to the weight. A simple perceptron network can be shown in Figure 2.2. The output of the example perceptron is illustrated in formula (2.1), where y is the output, x_i is the input, w_i is the weight of the neuron i, and f is the activation function.

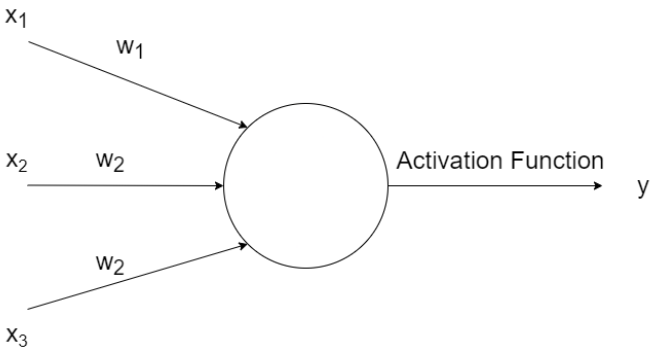

Figure 2.2: A Example of Simple Perceptron [4]

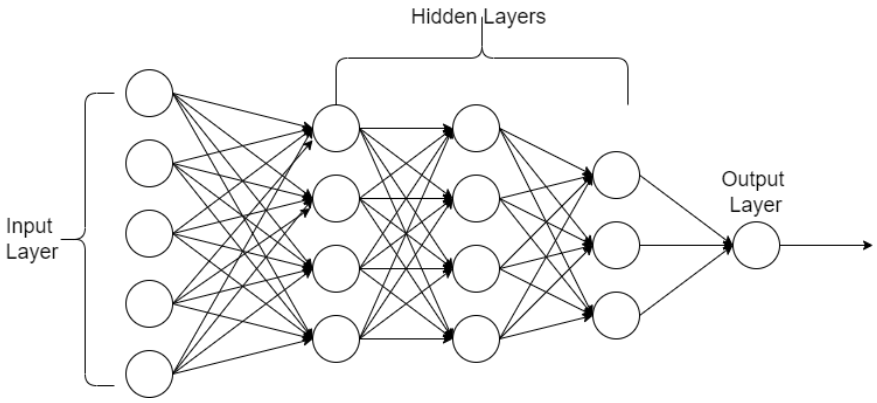

Figure 2.3: An Example of MLP [5]

$$y = f(\sum_{i=1}^{3} x_i \cdot w_i) \tag{2.1}$$

A single perceptron network has only one output layer, as shown in Figure 2.2. The inputs x_i where $i \in \{1, 2, 3\}$ can be considered as an input layer. If there are other layers between the input and output layer, the network is called MLP. These specific layers between the input and output layers are hidden layers.

During the training process, the weights and biases are initialized randomly. With the given inputs, the outputs are calculated by the perceptron algorithm. Then the gradient of the loss function is calculated by the difference between the actual output and the calculated output [28]. Afterward, the weight and the bias could be adjusted by a gradient descent optimization algorithm. The whole process is called back-propagation.

Figure 2.3 shows an example of an MLP model with one input layer, one output layer, and three hidden layers. If the number of hidden layers is more than one, the model could be considered as a DL architecture.

2.1.3 Activation Function

In a multiple-layer neural network, the input of the next layer is determined by the output of the current layer. A non-linear function is applied to represent the connection between two layers. This function is called the activation function, which is used to improve the fitting ability of the neural network. If there is no activation function between two layers, their connection will be a simple linear function. It means that multiple hidden layers will have the same effect as the simple perceptron. In my thesis, three activation functions are used, including Sigmod, Tanh, and Relu functions.

- Sigmod function is a usual linear activation function that is used to map consecutive real numbers into the interval[0,1] [16]. However, it is not as popular as before since it might bring about gradient explosion and gradient vanishing problems. Moreover, the output of the Sigmod function is not zero-centered. Formula (2.2) illustrates the Sigmod activation function.

$$Sigmod(x) = \frac{1}{1 + e^{-x}} \tag{2.2}$$

- Tanh function shown in (2.3) is applied to convert consecutive real numbers into the interval[-1,1]. And it solved the non-zero-centered problem of the Sigmod function.

$$Tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}} \tag{2.3}$$

- Relu is similar to a function to take the maximum value. When the input is smaller than zero, the output will be zero. When the input is larger than zero, the output will be itself. Differing from the previous

activation functions, the Relu function does not activate all the neurons simultaneously [29]. Furthermore, it can make the neural network sparse and more straightforward to compute [1]. The formula is shown in (2.4).

$$Relu(x) = max(0, x) \tag{2.4}$$

2.2 Reverse Engineering

In general, reverse engineering is the process of extracting and analyzing the system's working principle by deconstructing the software and hardware. For the software engineering aspect, reverse engineering can help developers to analyze and understand the internal architecture and details of the software to provide support in upgrading legacy systems or developing new and powerful software [30]. Moreover, it can maintain software security from plagiarism and hacking by extracting and analyzing the software principle. Hence, reverse engineering is a kernel process of software engineering.

2.2.1 Reverse Engineering for Deep Learning Models

With the development of computer science engineering, DL is widely used in many applications, such as image classification [37] or speech recognition [39]. Therefore, it is necessary to attach importance to the security of neural networks in these applications. Figure 2.4 shows the process of reverse engineering for deep learning models, containing two stages: the profiling stage and the attack stage. In the profiling stage, the attacker captures power traces during the execution of the target neural network. Then he/she uses traces and the label of traces to train a DL model. In the attack stage, the attacker captures traces as inputs of the trained DL model to acquire the parameters of the target neural network.

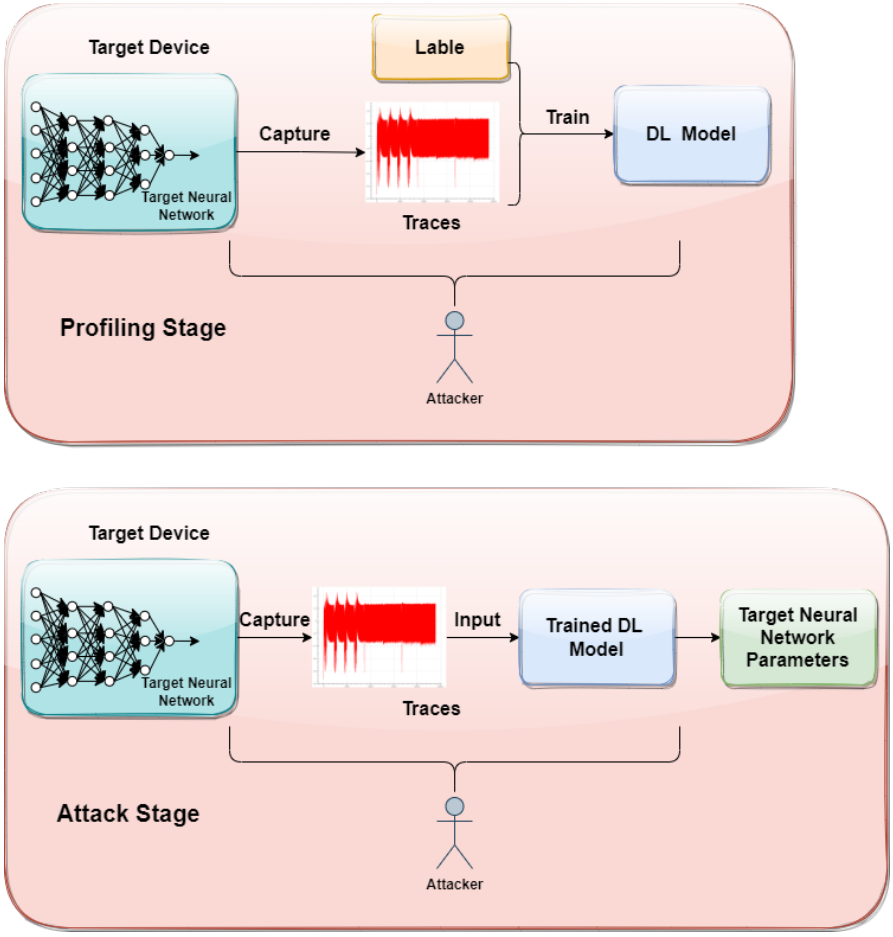

Figure 2.4: The process of reverse engineering for deep learning models

Recently, reverse engineering for deep learning models, especially for CNN, has achieved some gains through the SCA method. It is proved that both the network architecture and the weights of the CNN can be recovered only through the memory access pattern, the input, and the output of the accelerator, without internal access [17]. Furthermore, timing and EM-based SCA can also be used for reverse engineering of the MLP and CNN network model [5].

Chapter 3

Deep Learning Reverse Engineering on Neural Networks

3.1 Assumption

We assume that adversaries have a device that can execute different target neural networks with different weights, similar to the target device. For the training process, the adversary can control the inputs and parameters of the target neural network and capture the corresponding traces during the execution of the model. Furthermore, the parameters of the target neural network like weights and biases could be recorded by the adversary. However, for testing, the adversary can only access the device for a short time to capture a limited amount of power traces during the execution of the target neural network.

3.2 Experimental Setup

The overall experimental setup is shown in Figure 3.1, which consists of a ChipWhisperer Lite [18] and an 8-bit Atmel ATXmega128D4 microcontroller board [9]. The 8-bit Atmel ATXmega128D4 microcontroller board is used to load and execute the target neural network. The target neural network is implemented by C and compiled into a hex file. The ChipWhisperer Lite [18] is used to capture the power traces during the execution of the target neural network, which can be used for training and testing classifier models.

The system properties of the hardware are listed as follows:

- ChipWhisperer Lite [18]

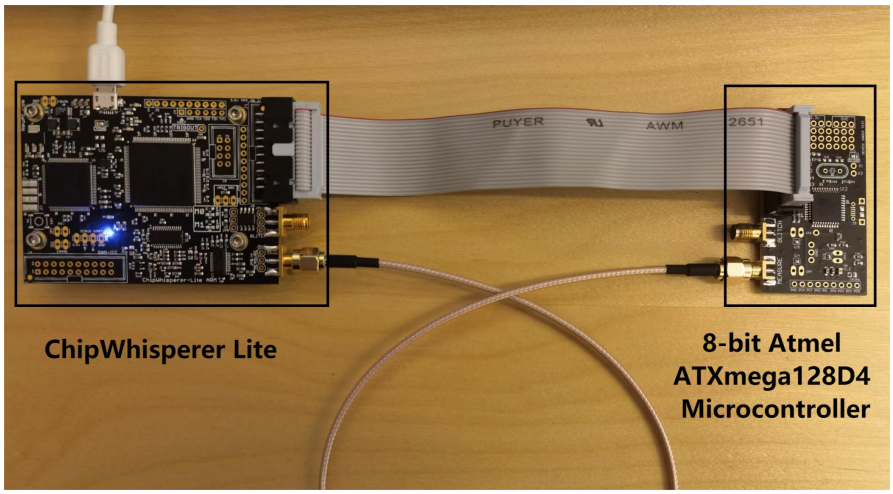

Figure 3.1: Experimental Setup

  - Built-in XMEGA and AVR programmers
  - Maximum sampling rate: 105MS/sec
  - Sample buffer size: 24537
  - Clock generation range: 5-200 MHz
  - Open-source

- 8-bit Atmel ATXmega128D4 board [9]
  - An 8-bit Atmel ATXmega128D4 microcontroller
  - Non-volatile program and data memories
  - Four channel event system
  - Four 16-bit timer/counters
  - Two USARTs

3.3 Target Neural Network Implementation

The target neural network for reverse engineering is implemented in C programming language and compiled into a hex program which can be executed in the 8-bit Atmel ATXmega128D4 microcontroller. The implementation of the target neural network is based on the open-source ChipWhisperer software toolkit, which provides a series of built-in functions. The kernel function of the implementation is 'simpleserial add command',

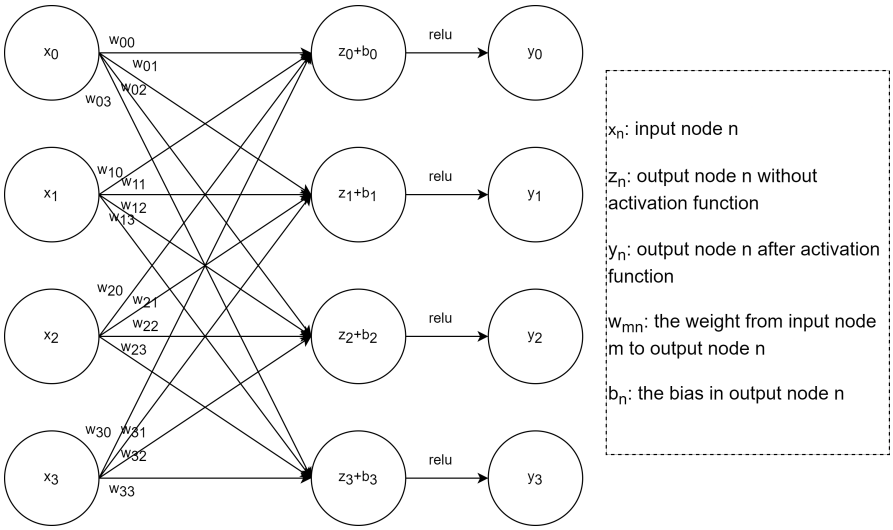

Figure 3.2: Target Neural Network Architecture

which can associate a simple command with a specific function. The programmer can control the victim board to execute the neural network by entering a command on the computer.

With $i \in \{0, 1, 2, 3\}$ and $j \in \{0, 1, 2, 3\}$, the target perceptron network is shown in Figure 3.2, where i means the input node i and j means the output node j. Due to the sample buffer size limitation of the ChipWhisperer Lite, the target neural network has only one input layer and one output layer. Each layer has four inputs x_i, and each input corresponds to a weight w_{ij}. The parameter b_i is the bias corresponding to the output node y_i, and y_i could be calculated as formula (3.1) and (3.2), where f is the activation function. In the activation function distinguishing experiment, we use Relu, Sigmod, and Tanh as our activation functions. While in the other experiments, only the Relu function is used to implement the target neural network.

$$z_i = \sum_{j=0}^{3} x_j \cdot w_{ij}, i \in \{0, 1, 2, 3\} \tag{3.1}$$

$$y_i = f(z_i + b_i), i \in \{0, 1, 2, 3\} \tag{3.2}$$

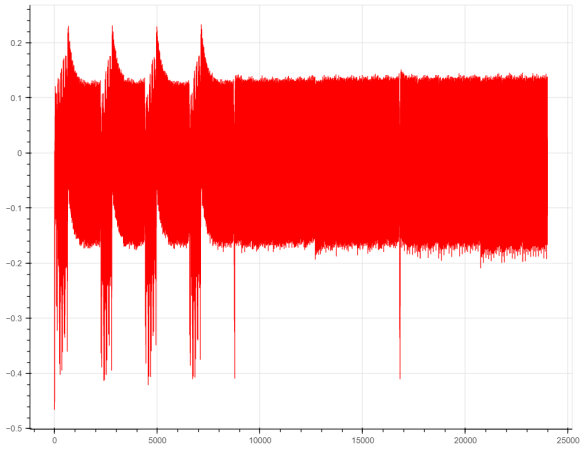

Figure 3.3: Trace of the target neural network

3.4 Trace Acquisition

During the experiments, we use $\vec{T}$ to denote power traces captured during the execution of the model.

In the first experiment, we capture the traces with different activation functions, which are the Relu, Sigmod, and Tanh functions, respectively. Then we observe the features of these traces to determine to which extent activation functions can be distinguished.

In the second experiment, traces are used for three parts of datasets: training, validation, and test sets. The training set is used to train the DL classifier, which is MLP in my thesis. The validation set is used for validating the MLP classifier during the training process. And the test set is used to evaluate the performance of the trained MLP classifier.

Figure 3.3 shows an example of how a trace captured from the 8-bit Atmel ATXmega128D4 microcontroller during the execution of the target neural network looks like. The variable w_{ij} denotes the weight from the input neuron i to the output neuron j and x_i means the input of input neuron i.

For the training and validation process, there are two sets of traces. In the first set, the number of traces is 250K and all of these traces are captured with random weights $w_{ij} \in \{0, 1, 2, ..., 255\}$ and random inputs $x_i \in \{0, 1, 2, ..., 255\}$. The Hamming weight in Section 3.6 of the attack point in Section 3.5 is distributed unbalanced, meaning that the amounts of each Hamming weight are unequal. In the second set, I control the value of the input x_i and the weight w_{ij} to balance the distribution of the Hamming weight. The number of traces in the dataset is 320K in total, and for each

Hamming weight value, there are 20k traces. In both datasets, 80 percent of traces are used for training, and the remaining traces are used for validation.

For the test process, there are also two sets of traces. The first set has 5000 traces with random inputs and fixed weights. In the second test set, there are 256 values of weights $w_{ij} = \{0, 1, 2..., 255\}$. For each value of weights, 2560 traces are captured with 256 different inputs $x_i = \{0, 1, 2..., 255\}$, and each value of the input is repeated ten times.

3.5 Attack Point

In the experiments, the most vulnerable point of the target neural network is variable t_{ij} which is the multiplication of the weight w_{ij} and the input x_i, with $i \in \{0, 1, 2, 3\}$ and $j \in \{0, 1, 2, 3\}$ according to the leakage detection results. The algorithm of the attack point can be illustrated in the formula (3.3). We focus on the multiplication of the first weight and first input in formula (3.4), and others will be the same.

$$t_{ij} = w_{ij} \cdot x_j, i \in \{0, 1, 2, 3\}, j \in \{0, 1, 2, 3\} \tag{3.3}$$

$$t_{00} = w_{00} \cdot x_0 \tag{3.4}$$

3.6 Power Model

Due to the limitations of the 8-bit Atmel ATXmega128D4 microcontroller, the input x_i and the weight w_{ij} are both 8-bit variables in my implementation, where $i \in \{0, 1, 2, 3\}$ and $j \in \{0, 1, 2, 3\}$. To simplify the computation, the input and the weight can be considered as 8-bit integers $x_i \in \{0, 1, 2, ..., 255\}$ and $w_{ij} \in \{0, 1, 2, ..., 255\}$. In Section 3.5, it shows the attack point $t_{ij} \in \{0, 1, 2, ..., 65535\}$, which is the multiplication of the weight and the input, while $i \in \{0, 1, 2, 3\}$ and $j \in \{0, 1, 2, 3\}$. However, to train a 65536-output classifier (MLP), we need to spend a few months capturing power traces. Therefore, it is essential to decrease the number of outputs.

A power model can map the original value to a new value by a specific algorithm. Hamming weight [34] is a typical power model, which means the number of non-zero bits of a vector. During my experiments, the Hamming weight power model shown in formula (3.5) is the Hamming weight of the attack point t_{ij}, where $i \in \{0, 1, 2, 3\}$ and $j \in \{0, 1, 2, 3\}$.

$$Label_{Hwij} = HW(w_{ij} \cdot x_i), i \in \{0,1,2,3\}, j \in \{0,1,2,3\} \quad (3.5)$$

3.7 Leakage Detection

After the traces acquisition, TVLA analysis is applied to detect the location of the leakage point. In the beginning, I try to use TVLA analysis to detect whether the weight of the target neural network has enough leakage or not. The traces are partitioned into two sets, T_0 and T_1. The weights of traces in T_0 are smaller than or equal to 128, while those in T_1 are larger than 128.

TVLA analysis is also used to detect if there is leakage of the attack point t_{00}. Similarly, as before, the traces are divided into two sets, T_0 and T_1. The attack points t_{00} of the traces in T_0 are smaller than or equal to 12000, while those in T_1 are larger than 12000.

Section 3.6 defines the label of the Hamming weight power model in the first neuron to simplify the experiment. There are two sets of traces, T_0 and T_1, for the TVLA analysis. In the first set, the label $Label_{Hw00}$ is smaller than or equal to 6, while in the second set, the label $Label_{Hw00}$ is larger than 6. All the results of three TVLA analysis experiments are illustrated in Section 4.2.

3.8 Training Parameters

Based on the results of TVLA analysis in Section 4.2, the classifier (MLP) is trained on traces with labels made by Hamming weight power model $Label_{Hw00}$. During the training stage, two types of MLP models are trained for reverse engineering.

The first one is MLP with binary output, called MLP_2 which is used to distinguish the label $Label_{Hw00} = i$ or $Label_{Hw00} \neq i$ where $i \in \{0,1,2,...,15\}$. First, we select all the traces with $Label_{Hw00} = i$ from the 250K traces, and then the same number of traces with $Label_{Hw00} \neq i$ are randomly selected from the dataset with 250K traces. Eventually, we mix the two sets of traces and do a shuffle to get a new training set. When $i = 3$, we get the best MLP_2 with label $Label_{Hw00} = 3$ and $Label_{Hw00} \neq 3$, shown in Table 3.1. The input size of MLP_2 is 200 for the trace segment with the interval[0:200], which will be illustrated in Section 4.2. The batch size is 256, and the number of epochs is 100 for training MLP_2. The Nadam optimizer is used to train MLP_2 with a learning rate of 0.00003 and constant epsilon of 1e-

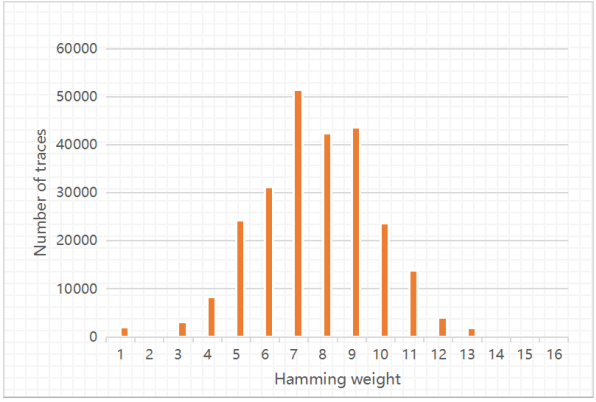

Figure 3.4: The distribution of Hamming weight in the dataset with 250K traces

08. In total, 16640 traces with labels $Label_{Hw00} = 3$ and $Label_{Hw00} \neq 3$ are used to train MLP_2, where 80% of traces are randomly selected for training, and 20% are for validation.

Layer type	Output shape	# Parameters
Batch Normalization	200	800
Dense 1	256	51456
Batch Normalization	256	1024
Dropout	256	0
Softmax	2	514
Total parameters:	53794	
Trainable parameters:	52882	

Table 3.1: The architecture of MLP_2 with label $Label_{Hw00} = 3$ and $Label_{Hw00} \neq 3$.

Another type of classifier is the 16-output classifier (MLP) called MLP_{16}. Firstly, 250K traces are captured with the random inputs and weights to train MLP_{16}. Nevertheless, the distribution of the Hamming weight in 250K traces is unbalanced, which is shown in Figure 3.4. With the unbalanced dataset with 250K traces, the best structure of MLP_{16} is illustrated in table 3.2. Among the traces in the dataset, 80% are used for training, and 20% are for validation. The epoch and batch size are as the same as MLP_2. We use the Nadam optimizer with a new learning rate of 0.00004.

Since MLP_{16} trained on the unbalanced dataset performs poorly, I control the specific inputs and weights to capture the traces appropriately. A new

Layer type	Output shape	# Parameters
Batch Normalization	200	800
Dense 1	256	51456
Dropout	256	0
Softmax	16	4112
Total parameters:	56368	
Trainable parameters:	55968	

Table 3.2: The architecture of MLP_{16} trained with label $Label_{Hw00} \in \{0, 1, 2, ..., 15\}$ on unbalanced dataset with 250K traces

dataset with the balanced distribution of the Hamming weight is acquired. The total size of the dataset is 320K, and the number of traces with each Hamming weight $Label_{Hw00} \in \{0, 1, 2, ..., 15\}$ is 20k. Table 3.3 displays the final architecture of MLP_{16} with label $Label_{Hw00} \in \{0, 1, 2, ..., 15\}$ trained on the balanced dataset with 320K traces. Except for the learning rate of 0.00001, all training parameters are identical to the two previous classifiers.

Layer type	Output shape	# Parameters
Batch Normalization	200	800
Dense 1	256	51456
Dense 2	256	65792
Batch Normalization	256	1024
Relu	256	0
Dropout	256	0
Softmax	16	4112
Total parameters:	123184	
Trainable parameters:	122272	

Table 3.3: The architecture of MLP_{16} trained with label $Label_{Hw00} \in \{0, 1, 2, ..., 15\}$ on balanced dataset with 320K traces.

3.9 Evaluations

After training the classifiers (MLP), the next step is to test the performance of the trained classifiers. The multiple-trace attack method is utilized to reverse the weight of the target neural network, and the rank function is used for evaluation.

The test dataset contains traces representing the power consumption of the

execution of the same target neural network and different inputs. When one trace in the test set is fed into the trained MLP_{16}, the classifier generates a 16×1 array a_{HW} representing the probabilities of 16 labels $Label_{Hw00} \in \{0, 1, 2, ..., 15\}$. For each trace in the test set, the corresponding input of the target neural network is a given constant value while the weight is an unknown constant value. Hence, we assume that the weight $w_{00} \in \{0, 1, 2, ..., 255\}$ has 256 possibilities. With the determinate input x_0, the Hamming weight power model of the attack point $Label_{Hw00}$ could be computed, and a 256×1 array a_w is generated, which means the 16 Hamming weight values distributed on 256 different weights $w_{00} \in \{0, 1, 2, ..., 255\}$. Since each Hamming weight value corresponds to probabilities of 16 labels, the array a_w can be mapped into a new 256×1 array a_{wnew} which represents the probabilities of 16 labels distributed on 256 different weights $w_{00} \in \{0, 1, 2, ..., 255\}$.

The rank method is applied by comparing the actual weight w_{00true} with the array a_{wnew} to check the rank of the actual weight w_{00true} among all possibilities of the weight. With the multiple-trace attack method, plenty of 256×1 arrays a_{wnew} are acquired. Then, we take the logarithm operation of these arrays and sum them together to get the new prediction array a_{wsum}. Afterward, comparing the actual weight w_{00true} with the new prediction array a_{wsum}, the rank of the actual weight can be obtained. Finally, we record the number of traces when the rank of the actual weight reaches zero, meaning that we can reverse the weight of the target neural network using these numbers of traces.

In the actual scenario, the attacker only needs to capture a specific number of traces within one minute during the execution of the target neural network. Afterward, he/she can use the trained MLP_{16} to get the final prediction array. The weight among the 256 categories with the largest probabilities is likely the actual weight.

Chapter 4

Experimental Results

This section displays the experimental results, including the activation function distinguishing, leakage detection, training, and evaluation results.

4.1 Activation Function Results

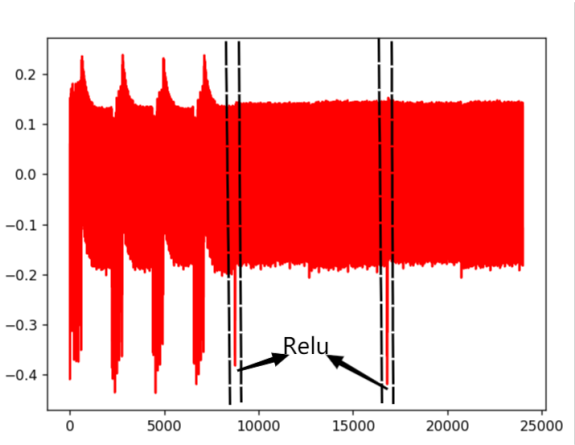

Figure 4.1: Trace of the target neural network with Relu activation function

Figure 4.1, Figure 4.2, and Figure 4.3 illustrate the traces of the target neural network with Relu, Sigmod, and Tanh activation functions, respectively. From the features of these three traces, we could distinguish each activation function clearly.

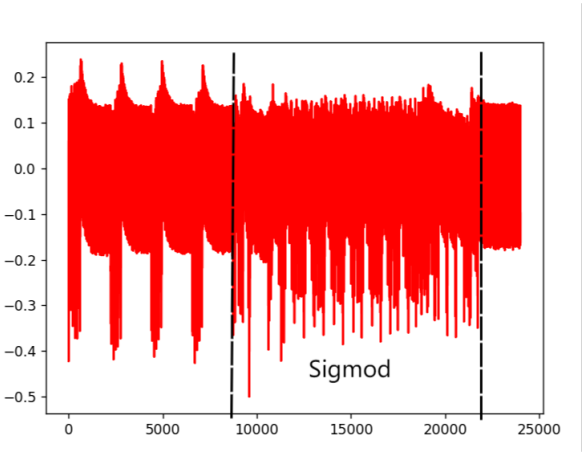

Figure 4.2: Trace of the target neural network with Sigmod activation function

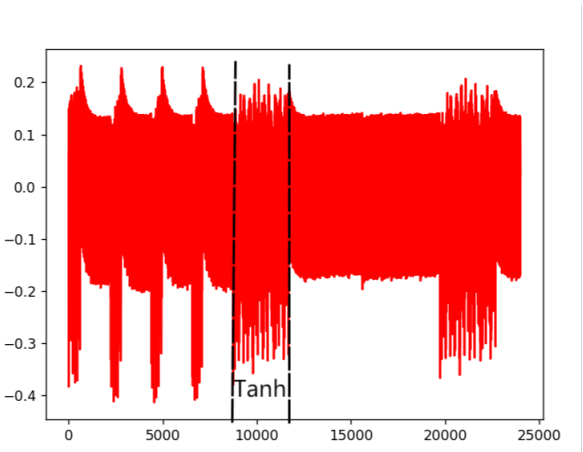

Figure 4.3: Trace of the target neural network with Tanh activation function

4.2 Leakage Detection Results

After capturing the traces from the 8-bit Atmel ATXmega128D4 microcontroller, we perform the TVLA analysis to detect whether the leakage occurs and where to determine the leakage interval. Three experiments are done for the TVLA analysis: weight w_{00}, the attack point t_{00} shown in Section 3.5, and the Hamming weight power model of the attack point $Label_{HW00}$ illustrated in Section 3.6.

Figure 4.4 illustrates the TVLA analysis result of sets with the weight $w_{00} \leq 128$ and $w_{00} > 128$. The maximum T-test value is 2.9337. Since the maximum T-test value is meager and there are no apparent peaks, we believe

that the weight w_{00} has nearly no leakage.

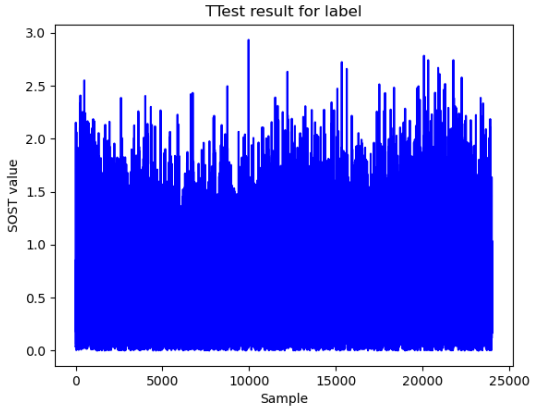

Figure 4.4: T-test results for the weight w_{00}

The T-test result of the attack point t_{00} is shown in Figure 4.5. Four obvious peaks represent the four similar multiplication operations: $t_{i0} = w_{i0} \times x_0, i \in \{0, 1, 2, 3\}$. The first peak has the maximum Sum of Squared pairwise Tdifferences (SOST) value of 43.3267 at point 98. By zooming into the interval[0:200], two peaks can be seen at points 20 and 98 in Figure 4.6.

As is known before, the weight $w_{00} \in \{0, 1, 2, ..., 255\}$ and the input $x_0 \in \{0, 1, 2, ..., 255\}$ are determined. Therefore, if the attack point t_{00} is used as the label to train a classifier (MLP), the number of the outputs will be 65535. We assume that each label requires 1000 traces to train the classifier, and the size of the total training set will be 65535k. It is impractical to capture such a large number of traces in this project.

Figure 4.7 illustrates the TVLA analysis result of the Hamming weight of attack point $Label_{HW00}$. The maximum SOST value is 44.7960 at point 20. To locate the specific leakage interval of the trace, we zoom the trace into the interval[0:200] in Figure 4.8. There are two peaks at points 20 and 98, which are the same as Figure 4.6. If we use $Label_{HW00}$ as a label to train a classifier, the number of labels is only 16. Finally, the trace with the interval[0:200] is selected to train the MLP classifier in the next stage.

4.3 Training Process Results

With the leakage detection results, we train a binary classifier MLP_2 and a 16-output classifier MLP_{16} with the traces in the interval[0:200] and $Label_{HW00}$.

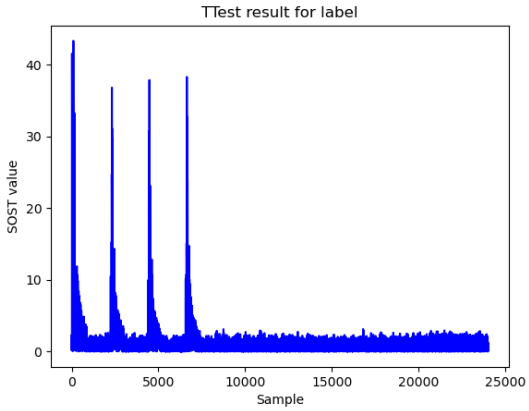

Figure 4.5: T-test results of traces partitioned by the attack point t_{00}

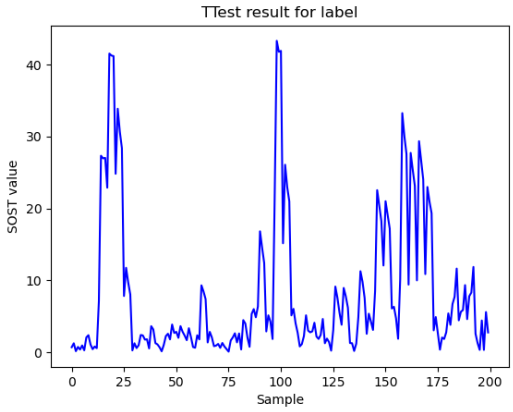

Figure 4.6: The zoomed interval [0:200] of Figure 4.5.

In Section 4.3.1, we use the dataset, which contains 250K traces acquired from the 8-bit Atmel ATXmega128D4 microcontroller for model training and validation. The distribution of the dataset is shown in Figure 3.4. And the test set contains 5k traces captured with random input and fixed weight $w_{00} = 121$.

In Section 4.3.2, we get the new training and validation dataset from the previous dataset with 250K traces, as explained in Section 3.8. In the test set, we capture 5k traces with random input and fixed weight $w_{00} = 253$.

While in Section 4.3.3, 320K traces are used to train and validate MLP_{16}. For the test process, we divide the test set into 256 groups with weight $w_{00} = i, i \in \{0, 1, 2, ..., 255\}$. Each group contains 2560 traces captured with inputs $x_0 = i, i \in \{0, 1, 2, ..., 255\}$, and each input has ten repetitions.

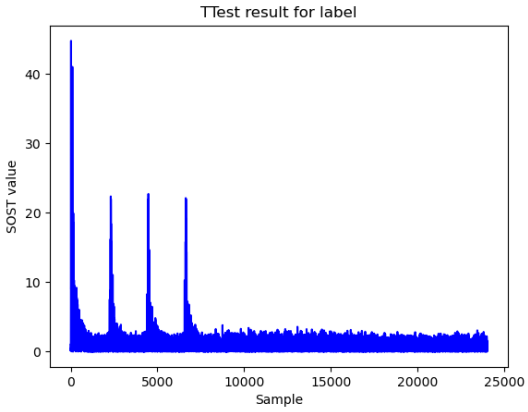

Figure 4.7: T-test results of traces partitioned by the Hamming weight of the attack point $Label_{HW00}$

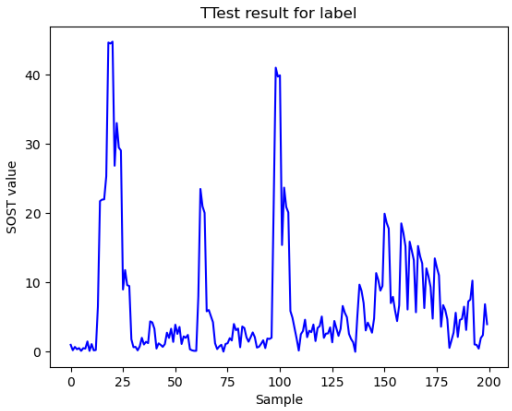

Figure 4.8: The zoomed interval [0:200] of Figure 4.7.

4.3.1 16-Output Classifier with Unbalanced Data

Firstly, we train MLP_{16} by using the unbalanced dataset with 250K traces. The dataset has 16 labels $Label_{HW00} = i$ where $i \in \{0, 1, 2, ..., 15\}$. In Figure 4.11, the final training and validation accuracy are 0.2362 and 0.2311, respectively. And the test accuracy is 0.2256. Considering that the random guess accuracy is 0.2055, this means the performance of MLP_{16} is unsatisfied.

4.3.2 Binary Classifier

For the binary classifier MLP_2, there are two labels in the dataset: $Label_{HW00} = i$ and $Label_{HW00} \neq i$ where $i \in \{0, 1, 2, ..., 15\}$. Since the

Model (i)	Training Loss	Validation Loss
3	0.5869	0.5822
4	0.6340	0.6437
5	0.6818	0.6799
6	0.6863	0.6886
7	0.6825	0.6823
8	0.6672	0.6662
9	0.6551	0.6503
10	0.6602	0.6479

Table 4.1: The loss results of MLP_2 with label $Label_{HW00} = i$ and $Label_{HW00} \neq i$ where $i \in \{3, 4, 5, 6, 7, 8, 9, 10\}$.

Model (i)	Training Accuracy	Validation Accuracy	Test Accuracy
3	0.7004	0.6986	0.6942
4	0.6442	0.6320	0.6191
5	0.5742	0.5662	0.5431
6	0.5529	0.5434	0.5388
7	0.5551	0.5418	0.5481
8	0.5736	0.5680	0.5618
9	0.6097	0.6079	0.5887
10	0.6229	0.6138	0.6395

Table 4.2: The training, validation and test accuracy of MLP_2 (MLP) trained with label $Label_{HW00} = i$ and $Label_{HW00} \neq i$ where $i \in \{3, 4, 5, 6, 7, 8, 9, 10\}$.

number of traces with labels $Label_{HW00} = i$ where $i \in \{0, 1, 2, 11, 12, 13, 14, 15\}$ is too few to train a binary classifier, we only train eight binary classifiers (MLP). The loss and accuracy results of training, validation, and the test are shown in tables 4.1 and 4.2. When $i = 3$, we get the best MLP_2. The accuracy and loss results are shown in Figure 4.14.

4.3.3 16-Output Classifier with Balanced Data

In this part, we control inputs and weights to balance the labels of the traces. The total size of the new dataset is 320K, and the number of traces for each Hamming weight is 20k. Figure 4.17 illustrates the training and validation loss and accuracy of MLP_{16} trained on the balanced dataset with 320K traces. The training and validation accuracy are 0.9430 and 0.9685, respectively, which are much higher than MLP_{16} trained on the unbalanced dataset. Then we use the

test set with $w_{00} = 31$ to test MLP_{16} and acquire the test accuracy of 0.7148.

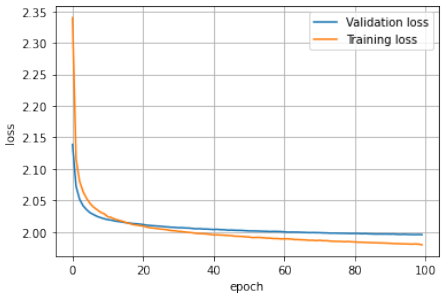

Figure 4.9: Loss

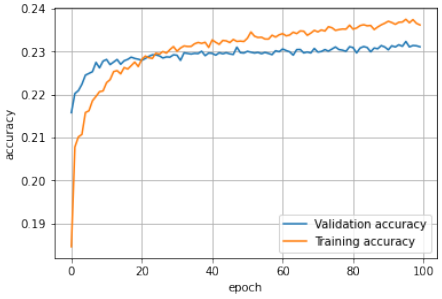

Figure 4.10: Accuracy

Figure 4.11: The loss and accuracy for model $Label_{HW00} = i$ where $i \in \{0, 1, 2, ..., 15\}$ trained on the unbalanced dataset

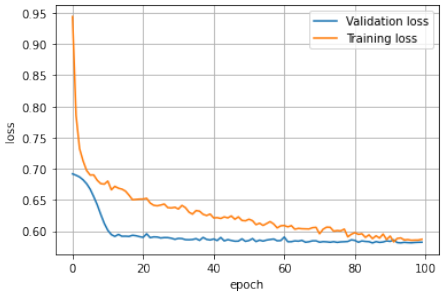

Figure 4.12: Loss

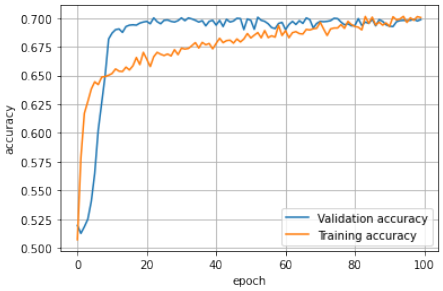

Figure 4.13: Accuracy

Figure 4.14: The loss and accuracy for model $Label_{HW00} = 3$ and $Label_{HW00} \neq 3$

4.4 Evaluation Results

After training different classifiers, we use the rank function and the multiple-trace attack method to examine whether the weight w_{00} could be reversed by the trained classifiers. Firstly, we reverse the weight w_{00} by using MLP_{16} trained on 250K unbalanced traces. Unfortunately, the rank of the correct weight goes up as the number of traces increases. Hence, the weight will never be reversed by this classifier.

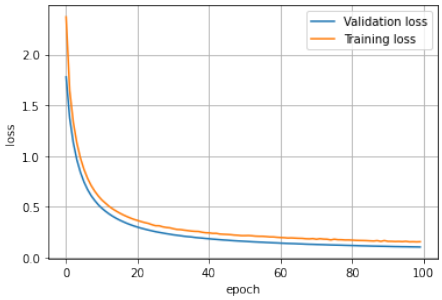

Figure 4.15: Loss

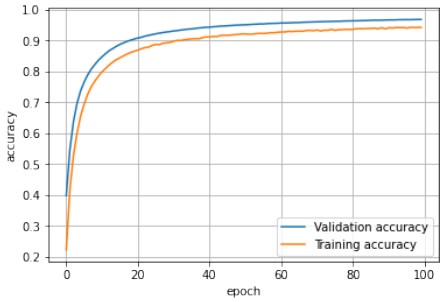

Figure 4.16: Accuracy

Figure 4.17: The loss and accuracy for model $Label_{HW00} = i$ where $i \in \{0, 1, 2, ..., 15\}$ trained on balanced dataset

4.4.1 Binary Classifier

For binary classifier MLP_2 trained with label $Label_{HW00} = i$ where $i \in \{5, 6, 7, 8\}$, we use the test set with $w_{00} = 253$ for the multiple-trace attack. Figure 4.22 shows the rank results of the different binary classifiers trained with label $Label_{HW00} = i$ for $i \in \{5, 6, 7, 8\}$. In these figures, abscissa means the number of traces, while ordinate means the rank of weight w_{00}.

As the number of traces increases, the rank of most of the binary classifiers first rise and then fall. This means for most of the binary classifiers, the weight w_{00} might be reversed with a large number of traces. However, in real attack scenarios, an attacker is assumed to have very limited time to access the victim device. Therefore, it is unrealistic to reverse the weight in the real scenario by our binary classifiers.

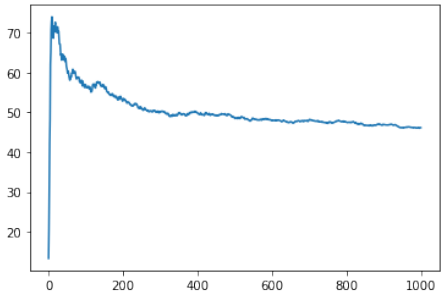

Figure 4.18: $Label_{HW00} = 5$

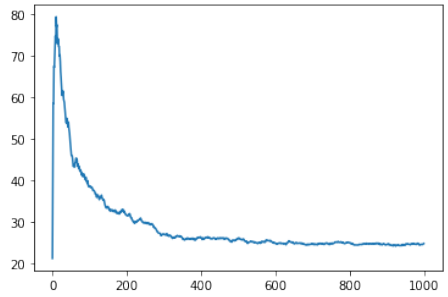

Figure 4.19: $Label_{HW00} = 6$

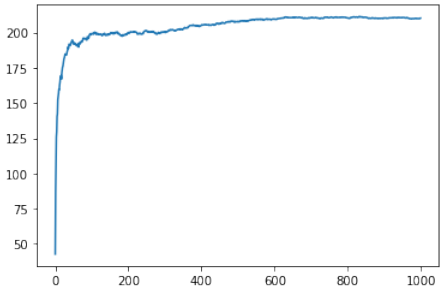

Figure 4.20: $Label_{HW00} = 7$

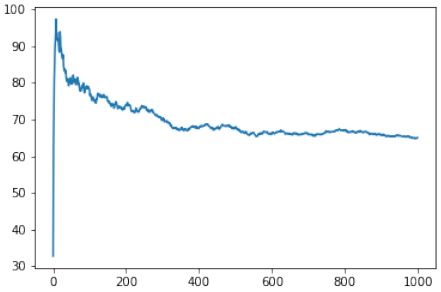

Figure 4.21: $Label_{HW00} = 8$

Figure 4.22: The rank of weight w_{00} for model trained with $Label_{HW00} = i$ and $Label_{HW00} \neq i$ where $i \in \{5, 6, 7, 8\}$

4.4.2 16-Output Classifier with Balanced Data

For the 16-output classifier MLP_{16} trained on the balanced dataset with 320K traces, we use 256 different test sets with weight $w_{00} = i$ where $i \in \{0, 1, 2, ..., 255\}$ to evaluate whether the weight w_{00} could be reversed or not. In each test set, 2560 traces are captured with fixed weight w_{00} and random inputs. For each input, we repeat the operation ten times.

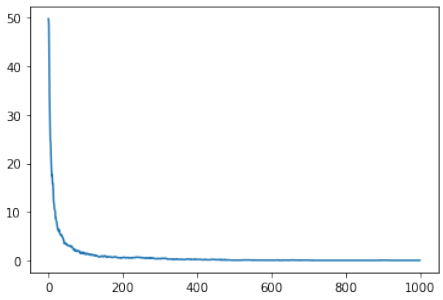

Figure 4.23: $w_{00} = 3$

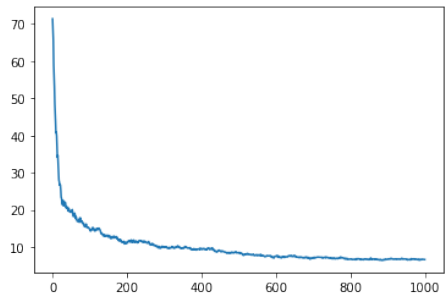

Figure 4.24: $w_{00} = 16$

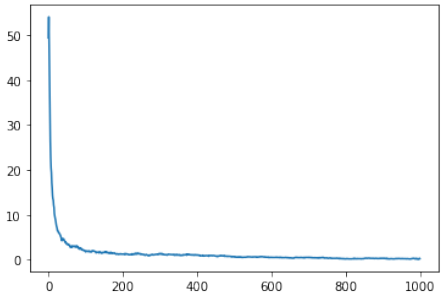

Figure 4.25: $w_{00} = 24$

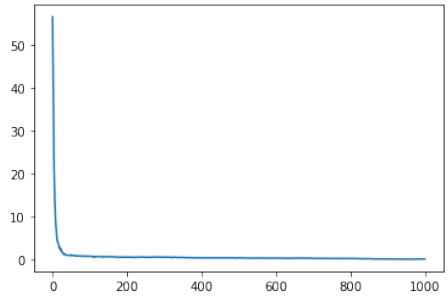

Figure 4.26: $w_{00} = 222$

Figure 4.27: The rank results weight $w_{00} = i$ used by model $Label_{HW00} = j$ where $i \in \{3, 16, 24, 222\}$, $j \in \{0, 1, 2, ..., 15\}$

From the 256 test sets, we randomly select 20 datasets for evaluation. For each dataset, we test how many traces are needed when the rank of weight reaches 0 for 100 times and record the number of average traces needed. Figure 4.27 illustrates the rank results of weight $w_{00} = i$ where $i \in \{3, 16, 24, 222\}$. All the rank values decrease as the number of traces increases and reach 0 at last. Table A.1 in Appendix shows the average number of traces needed to reverse different weight w_{00} by MLP_{16} trained with label $Label_{HW00} = i$ where $i \in \{0, 1, 2, ..., 15\}$. All the weights could be reversed within 1000 traces. For some specific weights like 0, it only needs one trace to reverse.

Chapter 5

Conclusions and Future Work

Initially, we explore the effect of the deep learning method and side-channel analysis on the reverse engineering of neural networks, and the conclusion is shown in section 5.1. Finally, the future work is displayed in section 5.2.

5.1 Conclusions

In this thesis, DL techniques and SCA were applied together in the reverse engineering of neural networks. All the experiments were conducted on an 8-bit Atmel ATXmega128D4 microcontroller. Firstly, we implemented a perceptron network model with 8-bit parameters as our target neural network on the victim chip.

Afterward, by analyzing the power consumption traces captured from the victim implementation, we show that it is feasible to recognize different activation functions used by the target neural network.

Furthermore, by using the trained MLP model to classify traces captured from the victim implementation, we accomplished reversing the weight of the target neural network with less than 1000 traces. We also show that, for some specific value of weights, the trained DL model is able to use only one trace on average to recover it.

5.2 Future Work

The potential future works of this project are listed as follows:

1. In our experiments, the weights of the target neural network are set to 8 bits. It is interesting to explore how DLSCA can be used to reverse the

neural network architecture with 32-bit parameters.

2. Since the target neural network in our experiment is a simulated perceptron network, it is necessary to explore the performance of DLSCA on reversing the architecture of an actual neural network. Moreover, it is possible to reverse CNN as the target neural network.

3. It is necessary to reverse more parameters of the neural network, like the number of layers and biases in the future.

References

[1] Jorge Albericio et al. “Cnvlutin: Ineffectual-Neuron-Free Deep Neural Network Computing”. In: *SIGARCH Comput. Archit. News* 44.3 (June 2016), pp. 1–13. ISSN: 0163-5964. DOI: 10.1145/3007787.3001138. URL: https://doi.org/10.1145/3007787.3001138.

[2] *Artificial intelligence*. eng. Clog ; [16]:2018. 2018. ISBN: 9780990422457.

[3] Giuseppe Ateniese et al. “Hacking Smart Machines with Smarter Ones: How to Extract Meaningful Data from Machine Learning Classifiers”. In: *International Journal of Security and Networks* 10 (June 2013). DOI: 10.1504/IJSN.2015.071829.

[4] *Basics of Multilayer Perceptron – A Simple Explanation of Multilayer Perceptron*. https://kindsonthegenius.com/blog/basics-of-multilayer-perceptron-a-simple-explanation-of-multilayer-perceptron/.

[5] L Batina et al. “CSI NN: Reverse Engineering of Neural Network Architectures Through Electromagnetic Side Channel”. eng. In: (2019), pp. 1–19.

[6] Jakub Breier et al. “SNIFF: Reverse Engineering of Neural Networks With Fault Attacks”. eng. In: *IEEE transactions on reliability* (2021), pp. 1–13. ISSN: 0018-9529.

[7] Akshay L Chandra. *McCulloch-Pitts Neuron — Mankind’s First Mathematical Model Of A Biological Neuron*. https://towardsdatascience.com/mcculloch-pitts-model-5fdf65ac5dd1.

[8] *Computer Vision – ACCV 2018 14th Asian Conference on Computer Vision, Perth, Australia, December 2–6, 2018, Revised Selected Papers, Part III*. eng. 1st ed. 2019.. Image Processing, Computer Vision, Pattern Recognition, and Graphics ; 11363. 2019. ISBN: 3-030-20893-1.

[9] ATMEL [ATMEL Corporation]. *ATXMEGA128D4 Datasheet*. URL: https://www.alldatasheet.com/datasheet-pdf/pdf/444349/ATMEL/ATXMEGA128D4.html.

[10] *Cryptography*. eng. HighBridge Audio, 2020. ISBN: 1-68457-913-9.

[11] Hanwen Feng, Weiguo Lin, and Wenqian Shang. "MLP and CNN-based Classification for Points of Interest in Side-Channel Attacks". In: *2019 IEEE/ACIS 18th International Conference on Computer and Information Science (ICIS)*. 2019, pp. 456–460. DOI: 10.1109/ICIS46139.2019.8940169.

[12] Matthew Fredrikson et al. "Privacy in Pharmacogenetics: An End-to-End Case Study of Personalized Warfarin Dosing". In: *23rd USENIX Security Symposium (USENIX Security 14)*. San Diego, CA: USENIX Association, Aug. 2014, pp. 17–32. ISBN: 978-1-931971-15-7. URL: https://www.usenix.org/conference/usenixsecurity14/technical-sessions/presentation/fredrikson_matthew.

[13] Daniel Genkin, Adi Shamir, and Eran Tromer. "Acoustic Cryptanalysis". eng. In: *Journal of cryptology* 30.2 (2016), pp. 392–443. ISSN: 0933-2790.

[14] Ran Gilad-Bachrach et al. "CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy". In: *Proceedings of The 33rd International Conference on Machine Learning*. Ed. by Maria Florina Balcan and Kilian Q. Weinberger. Vol. 48. Proceedings of Machine Learning Research. New York, New York, USA: PMLR, 20–22 Jun 2016, pp. 201–210. URL: https://proceedings.mlr.press/v48/gilad-bachrach16.html.

[15] *Guide to Deep Learning Basics Logical, Historical and Philosophical Perspectives*. eng. 1st ed. 2020.. 2020. ISBN: 3-030-37591-9.

[16] Simon Haykin. *Neural Networks: A Comprehensive Foundation*. 2nd. USA: Prentice Hall PTR, 1998. ISBN: 0132733501.

[17] Weizhe Hua, Zhiru Zhang, and G. Edward Suh. "Reverse-Engineering CNN Models Using Side-Channel Attacks". eng. In: *IEEE design and test* 39.4 (2022), pp. 15–22. ISSN: 2168-2356.

[18] NewAE Technology Inc. *Chipwhisperer*. URL: https://www.newae.com/hardware/chipwhisperer.

[19] QUISQUATER J. J. “A new tool for non-intrusive analysis of smart cards based on electro-magnetic emissions : The SEMA and DEMA methods”. In: *Eurocrypt Rump Session, 2000* (2000). URL: https://cir.nii.ac.jp/crid/1572543024163490304.

[20] Dirmanto Jap, Marc Stöttinger, and Shivam Bhasin. “Support Vector Regression: Exploiting Machine Learning Techniques for Leakage Modeling”. In: *Proceedings of the Fourth Workshop on Hardware and Architectural Support for Security and Privacy*. HASP ’15. Portland, Oregon: Association for Computing Machinery, 2015. ISBN: 9781450334839. DOI: 10.1145/2768566.2768568. URL: https://doi.org/10.1145/2768566.2768568.

[21] Paul Kocher, Joshua Jaffe, and Benjamin Jun. “Differential Power Analysis”. eng. In: *Advances in Cryptology — CRYPTO’ 99*. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 1999, pp. 388–397. ISBN: 3540663479.

[22] Paul C. Kocher. “Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems”. eng. In: *Advances in Cryptology — CRYPTO ’96*. Vol. 1109. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 104–113. ISBN: 9783540615125.

[23] Liran Lerman et al. “Template Attacks vs. Machine Learning Revisited (and the Curse of Dimensionality in Side-Channel Analysis)”. In: *Constructive Side-Channel Analysis and Secure Design*. Ed. by Stefan Mangard and Axel Y. Poschmann. Cham: Springer International Publishing, 2015, pp. 20–33. ISBN: 978-3-319-21476-4.

[24] Taikang Liu and Yongmei Li. *Electromagnetic Information Leakage and Countermeasure Technique: Translated by Liu Jinming, Liu Ying, Zhang Zidong, Liu Tao*. eng. Singapore: Springer Singapore. ISBN: 9789811043512.

[25] Zdenek Martinasek, Petr Dzurenda, and Lukas Malina. “Profiling power analysis attack based on MLP in DPA contest V4.2”. In: *2016 39th International Conference on Telecommunications and Signal Processing (TSP)* (2016), pp. 223–226.

[26] Timothy Masters. *Deep Belief Nets in C++ and CUDA C: Volume 3: Convolutional Nets*. eng. Apress, 2018. ISBN: 9781484237212.

[27] James Miller. *IBM Watson projects : eight exciting projects that put artificial intelligence into practice for optimal business performance*. eng. 1st edition. 2018. ISBN: 1-78934-669-X.

[28] Tom M. Mitchell. *Machine Learning*. New York: McGraw-Hill, 1997. ISBN: 978-0-07-042807-2.

[29] Vinod Nair and Geoffrey E. Hinton. "Rectified Linear Units Improve Restricted Boltzmann Machines". In: *ICML*. 2010.

[30] *Reverse Engineering*. eng. 1st ed. 1996.. 1996. ISBN: 0-585-27477-0.

[31] Frank Rosenblatt. *Principles of neurodynamics : perceptrons and the theory of brain mechanisms*. eng. Washington, 1962. ISBN: 991-522872-4.

[32] Erica Sadun. *Talking to Siri*. eng. 1st edition. 2012. ISBN: 0-13-305697-X.

[33] Mohit Sewak. *Practical Convolutional Neural Networks*. eng. 1st edition. 2018.

[34] Iouliia Skliarova. *FPGA-BASED Hardware Accelerators*. eng. 1st ed. 2019.. Lecture Notes in Electrical Engineering, 566. 2019. ISBN: 3-030-20721-8.

[35] *Speech Recognition*. eng. InTech. ISBN: 953-51-5753-1.

[36] Eran Tromer, Dag Arne Osvik, and Adi Shamir. "Efficient Cache Attacks on AES, and Countermeasures". eng. In: *Journal of cryptology* 23.1 (2009), pp. 37–71. ISSN: 0933-2790.

[37] Lizhe Wang et al. "Spectral–spatial multi-feature-based deep learning for hyperspectral remote sensing image classification". eng. In: *Soft computing (Berlin, Germany)* 21.1 (2016), pp. 213–221. ISSN: 1432-7643.

[38] Yong Xiao, Jin Xin, and Yanhua Shen. "CNN Based Electromagnetic Side Channel Attacks on SoC". eng. In: *IOP conference series. Materials Science and Engineering* 782.3 (2020), p. 32055. ISSN: 1757-8981.

[39] Dong Yu. *Automatic Speech Recognition A Deep Learning Approach*. eng. 1st ed. 2015.. Signals and Communication Technology. 2015. ISBN: 1-4471-5779-6.

[40] W Yu and J Chen. "Deep learning-assisted and combined attack: a novel side-channel attack". eng. In: *Electronics letters* 54.19 (2018), pp. 1114–1116. ISSN: 0013-5194.

Appendix A

Test Results Table

Test Dataset (i)	Weight w_{00}	Number of Test	Average Traces Needed
1	0	100	1
2	2	100	56
3	3	100	551
4	9	100	156
5	13	100	135
6	16	100	885
7	22	100	167
8	24	100	995
9	39	100	364
10	53	100	59
11	70	100	64
12	82	100	36
13	94	100	102
14	121	100	171
15	153	100	33
16	189	100	132
17	222	100	944
18	230	100	31
19	243	100	690
20	251	100	328

Table A.1: The number of average traces needed to reverse different weight w_{00} for 16-label classifier (MLP) with label $Label_{HW00} = i$ where $i \in \{0, 1, 2, ..., 15\}$.

TRITA-EECS-EX-2023:21